



PROJE TASARIM RAPORU

Ders: BİL 204 / Yazılım Mühendisliği
Bölüm/Şube: Bilgisayar Mühendisliği
Grup Adı: BidLance
Proje Adı: Online Müzayede Sistemi
Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- Kübra Alkayış (24100011053)
- Huriye Dursun (24100011012)
- Elif Su Koçak (24100011015)
- Ayşe Sena Dişbudak (24100011010)
- Zeynep Nisa Koç (24100011008)
- Şeval Özkoç (24100011002)

İçindekiler

1. GİRİŞ (INTRODUCTION)	3
1.1. Sistemin Amacı	3
1.2. Problem Tanımı (Problem Definition)	3
1.3. Önerilen Çözüm (Proposed Solution)	3
1.4. Tasarım Hedefleri	4
1.4.1. Eş zamanlılıkta (Concurrency) Hatasız Teklif Yönetimi	4
1.4.2. Tam İzlenebilirlik (Traceability) ve Şeffaflık Sağlanması	4
1.4.3. Otomatik ve Güvenilir Müzayede Yönetimi	5
1.4.4. Modüler ve Genişletilebilir Sistem Mimarisi	5
2. ÜST DÜZEY YAZILIM MİMARİSİ	5
2.1. Alt Sistem Ayrıştırması	5
2.2. Donanım/Yazılım Eşlemesi	6
2.3 Kalıcı Veri Yönetimi	7
2.4. Erişim Kontrolü ve Güvenlik	8
2.5. Küresel Kontrol Akışı	9
2.6. Sınır Koşulları	9
2.6.1. Başlatma (Initialization)	10
2.6.2. Sonlandırma (Termination)	10
2.6.3. Hata Durumları (Error States)	10
3. ALT DÜZEY TASARIM	11
3.1. Nesne Tasarım Kararları	11
3.2. Son Nesne Tasarımı	12
3.3. Paketler / Katmanlar	13
3.3.1. Dış Paketler	13
3.3.2. İç Paketler (Uygulama - App Yapısı)	14
3.4. Sınıf Arayüzleri	14
3.4.1. Entity (Varlık) Sınıfları	14
3.4.2. Control (Kontrol / İş Mantığı) Sınıfları	16
3.4.3. Boundary (Arayüz / Sınır) Sınıfları	17
3.5. Tasarım Desenleri	18
Sözlük (Glossary)	19
Referanslar	20

1. GİRİŞ (INTRODUCTION)

1.1. Sistemin Amacı

Bu projenin temel amacı, geleneksel açık artırma süreçlerini dijital bir platforma taşıyarak kullanıcıların zaman ve mekân kısıtlaması olmaksızın güvenli, hızlı ve şeffaf bir şekilde ticaret yapabilmelerini sağlayan bir Online Müzayede Sistemi tasarlamaktır. Sistem; satıcıların ürünlerini yönetebildiği, müşterilerin rekabetçi bir ortamda teklif verebildiği ve yöneticilerin süreci denetleyebildiği bütünlüklü bir yapı sunmayı hedeflemektedir. Bu doğrultuda proje, yalnızca işlevsel bir uygulama geliştirmeyi değil, aynı zamanda modern yazılım mühendisliği prensiplerine uygun, sürdürülebilir ve ölçeklenebilir bir sistem ortaya koymayı amaçlamaktadır.

1.2. Problem Tanımı (Problem Definition)

Geleneksel müzayede sistemleri, fiziksel katılım gerekliliği, sınırlı erişim ve manuel işlem süreçleri nedeniyle birçok dezavantaj barındırmaktadır. Kullanıcıların belirli bir ortamda bulunma zorunluluğu, pazarın genişlemesini engellerken ürünlerin gerçek değerine ulaşmasını zorlaştırmaktadır. Ayrıca manuel süreçler, hata riskini artırmakta ve işlem takibini zorlaştırmaktadır.

Mevcut dijital platformlarda ise farklı teknik problemler ortaya çıkmaktadır. Özellikle eşzamanlı teklif verme durumlarında sistemlerin gelen talepleri doğru sıralayamaması, veri tutarsızlığına ve hatalı sonuçlara neden olabilmektedir. Bunun yanında teklif geçmişinin yeterince izlenebilir olmaması, kullanıcı güvenliğini zedeleyen önemli bir sorundur. Yönetici kontrol mekanizmalarının yetersiz olması da sistemde şeffaflık ve denetim eksikliğine yol açmaktadır.

Bu bağlamda, güvenli, tutarlı, şeffaf ve otomatik çalışan bir müzayede sistemine duyulan ihtiyaç açıkça ortaya çıkmaktadır.

1.3. Önerilen Çözüm (Proposed Solution)

Belirtilen problemleri çözmek amacıyla, Python tabanlı Django framework'ü kullanılarak geliştirilen ve nesne yönelimli programlama (OOP) prensiplerine dayanan web tabanlı bir sistem önerilmiştir. Sistem mimarisi, Django'nun "app" yapısı esas alınarak modüler şekilde tasarlanmıştır. Her bir işlevsel bileşen (kullanıcı yönetimi, ürün yönetimi, teklif sistemi vb.) ayrı bir uygulama (app) olarak ele alınmış ve bu yapı içerisinde nesne tabanlı sınıflar kullanılarak sistem davranışları modellenmiştir. Bu yaklaşım, sistemin bakımını kolaylaştırmakta, yeniden kullanılabilirliği artırmakta ve geliştirme sürecini daha yönetilebilir hale getirmektedir.

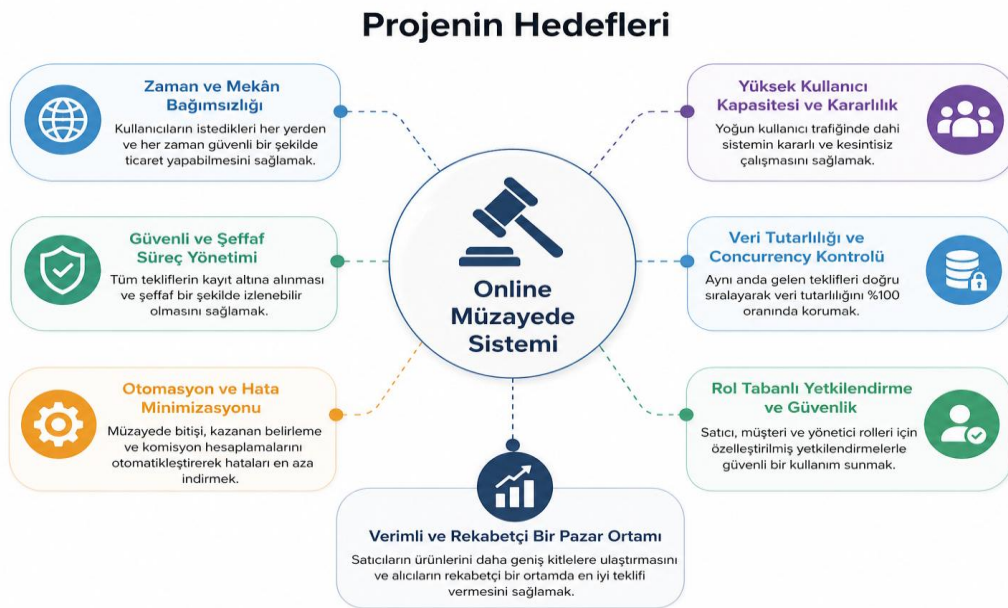
Önerilen sistemde her teklif, bağımsız bir nesne olarak ele alınmakta ve zaman damgası ile birlikte kaydedilmektedir. Bu sayede sistemde tam izlenebilirlik sağlanmakta ve geçmiş işlemler güvenli bir şekilde saklanmaktadır. Ayrıca veri tabanı seviyesinde uygulanacak kilitleme (locking) mekanizmaları sayesinde eşzamanlı teklif durumlarında veri tutarlılığı korunmaktadır.

Sistem, müzayede süresi dolduğunda otomatik olarak açık artırmayı sonlandırmakta, en yüksek teklifi veren kullanıcıyı belirlemekte ve komisyon hesaplamalarını gerçekleştirmektedir. Bu

otomasyon, işlem sürecini hızlandırırken insan hatasını minimum seviyeye indirmektedir. Ayrıca rol tabanlı erişim kontrolü (RBAC) mekanizması ile kullanıcı yetkileri güvenli bir şekilde yönetilmekte ve her kullanıcının yalnızca yetkili olduğu işlemleri gerçekleştirmesi sağlanmaktadır.

1.4. Tasarım Hedefleri

Geleneksel müzayede sistemleri, fiziksel katılım gerekliliği, sınırlı erişim ve manuel işlem süreçleri nedeniyle birçok dezavantaj barındırmaktadır. Kullanıcıların belirli bir ortamda bulunma zorunluluğu, pazarın genişlemesini engellerken ürünlerin gerçek değerine ulaşmasını zorlaştırmaktadır. Ayrıca manuel süreçler, hata riskini artırmakta ve işlem takibini zorlaştırmaktadır.



1.4.1. Eş zamanlılıkta (Concurrency) Hatasız Teklif Yönetimi

Bu projenin en kritik hedeflerinden biri, birden fazla kullanıcının aynı anda teklif verdiği durumlarda sistemin hatasız ve tutarlı bir şekilde çalışmasını sağlamaktır. Geleneksel ve birçok dijital sistemde, eşzamanlı işlemler veri çakışmalarına ve hatalı sonuçlara yol açabilmektedir. Bu projede ise, veri tabanı seviyesinde uygulanacak kontrol ve kilitleme mekanizmaları sayesinde tüm teklifler doğru sırayla işlenmekte ve veri tutarlılığı korunmaktadır. Böylece aynı anda gelen teklifler arasında adil bir sıralama sağlanmakta, birden fazla kazanan oluşması gibi kritik hatalar engellenmektedir. Bu yaklaşım, sistemin gerçek dünya koşullarına uygun, güvenilir ve mühendislik açısından güçlü bir altyapıya sahip olmasını sağlamaktadır.

1.4.2. Tam İzlenebilirlik (Traceability) ve Şeffaflık Sağlanması

Sistemin bir diğer önemli hedefi, tüm müzayede süreçlerinin şeffaf ve izlenebilir bir yapıda yürütülmesidir. Bu doğrultuda her teklif, bağımsız bir veri nesnesi olarak ele alınmakta ve zaman damgası ile birlikte kayıt altına alınmaktadır. Bu sayede kullanıcıların yaptığı tüm işlemler geriye dönük olarak incelenebilmekte ve sistemde herhangi bir veri kaybı veya manipülasyonu engellenmektedir. Teklif geçmişinin detaylı şekilde saklanması, hem

kullanıcılar arasında güven ortamı oluşturmakta hem de yöneticilerin sistem üzerindeki denetim gücünü artırmaktadır. Böylece sistem, yalnızca işlevsel bir platform olmanın ötesine geçerek şeffaf ve güvenilir bir dijital müzayede ortamı sunmaktadır.

1.4.3. Otomatik ve Güvenilir Müzayede Yönetimi

Bu projede müzayede sürecinin mümkün olan en yüksek seviyede otomatikleştirilmesi hedeflenmiştir. Sistem, belirlenen süre dolduğunda açık artırmayı otomatik olarak sonlandırmakta, en yüksek teklifi veren kullanıcıyı belirlemekte ve komisyon hesaplamalarını herhangi bir insan müdahalesine ihtiyaç duymadan gerçekleştirmektedir. Bu otomasyon sayesinde manuel işlem hataları ortadan kaldırılmakta, işlem süreci hızlandırılmakta ve sistem güvenilirliği artırılmaktadır. Ayrıca bu yapı, kullanıcı deneyimini iyileştirirken sistemin sürekli ve kesintisiz çalışmasını da desteklemektedir. Sonuç olarak, otomatik yönetim mekanizması projenin hem performansını hem de doğruluğunu artıran temel bileşenlerden biri olarak öne çıkmaktadır.

1.4.4. Modüler ve Genişletilebilir Sistem Mimarisi

Projenin bir diğer önemli hedefi, sürdürülebilir ve geliştirilebilir bir yazılım mimarisi oluşturmaktır. Bu amaçla sistem, Django framework'ünün sunduğu “app” yapısı kullanılarak modüler bir şekilde tasarlanmıştır. Her işlevsel bileşen ayrı bir uygulama olarak ele alınmış ve nesne yönelimli programlama prensipleri doğrultusunda modellenmiştir. Bu yapı sayesinde sistemin bakım süreçleri kolaylaşmakta, kod tekrarının önüne geçilmekte ve yeni özelliklerin sisteme entegrasyonu oldukça hızlı ve sorunsuz bir şekilde gerçekleştirilebilmektedir. Böylece proje, yalnızca mevcut ihtiyaçlara cevap veren bir sistem değil, aynı zamanda gelecekte genişletilebilir ve sürdürülebilir bir yazılım çözümü olarak konumlanmaktadır.

2. ÜST DÜZEY YAZILIM MİMARİSİ

Bu bölümde Online Müzayede Sistemi'nin genel yazılım mimarisi açıklanmaktadır. Sistem, web tabanlı bir yapı üzerine kurulmuş olup kullanıcıların tarayıcı üzerinden sisteme erişmesini sağlar.

Mimari yapı katmanlı olarak tasarlanmıştır. Bu yapı; kullanıcı arayüzü, iş mantığı ve veri yönetimi bileşenlerinin birbirinden ayrılmasını sağlar. Böylece sistem daha düzenli, sürdürülebilir ve geliştirilebilir hale gelir.

Sistem genel olarak Web Arayüzü (Frontend), Uygulama Katmanı (Backend) ve Veritabanı olmak üzere üç ana bileşenden oluşmaktadır.

2.1. Alt Sistem Ayrıştırması

Online Müzayede Sistemi, işlevlerine göre farklı alt sistemlere ayrılmıştır. Bu alt sistemler aşağıda açıklanmıştır:

1. Web Arayüzü (Frontend): Kullanıcıların sistemle etkileşime geçtiği katmandır. Kullanıcılar bu katman üzerinden kayıt olur, giriş yapar, ürünleri görüntüler ve teklif verir.

2. **Uygulama Katmanı (Backend):** Sistemin iş mantığının bulunduğu katmandır. Kullanıcıdan gelen istekler burada işlenir. Teklif kontrolü, kullanıcı yetkilendirme ve müzayede yönetimi gibi işlemler bu katmanda gerçekleştirilir.

3. **Veritabanı:** Sistemdeki tüm verilerin saklandığı katmandır. Kullanıcı bilgileri, ürünler, teklifler ve müzayede verileri burada tutulur.

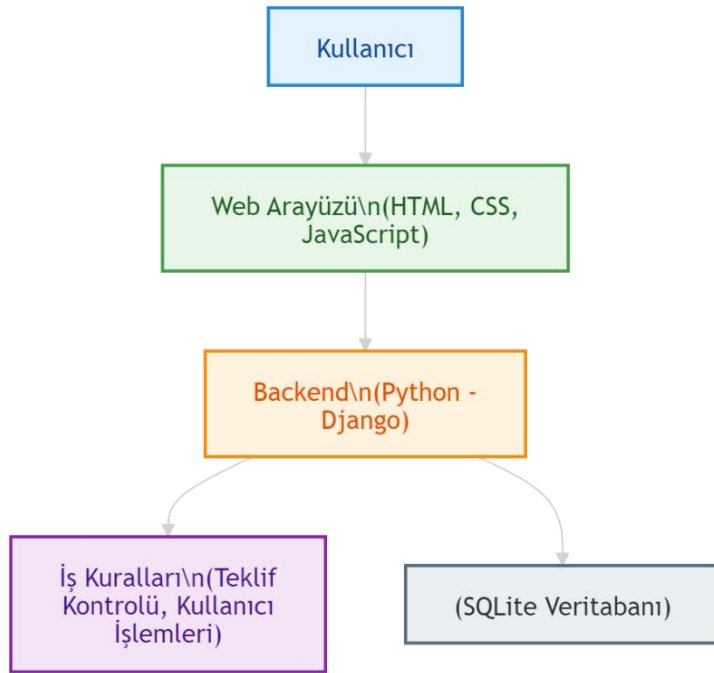


Figure 1: Sistem Mimari Diyagramı

Figure 1, sistemin frontend, backend ve veritabanı katmanlarından oluştuğunu göstermektedir. Kullanıcıdan gelen istekler frontend üzerinden backend'e iletilir. Backend katmanında iş kuralları uygulanır ve veriler SQLite veritabanında saklanır.

2.2. Donanım/Yazılım Eşlemesi

Online Müzayede Sistemi web tabanlı bir uygulama olarak tasarlanmıştır ve modern yazılım teknolojileri kullanılarak geliştirilmektedir.

Yazılım bileşenleri:

- Programlama dili olarak Python kullanılmaktadır.
- Backend geliştirme için Django framework'ü tercih edilmiştir.
- Frontend tarafında HTML, CSS ve JavaScript kullanılmaktadır.
- Veritabanı olarak SQLite (geliştirilebilir) kullanılmaktadır.

Donanım gereksinimleri:

Sistem, herhangi bir modern web tarayıcısı bulunan cihaz üzerinden çalıştırılabilir. Kullanıcıların ekstra bir donanıma ihtiyacı yoktur.

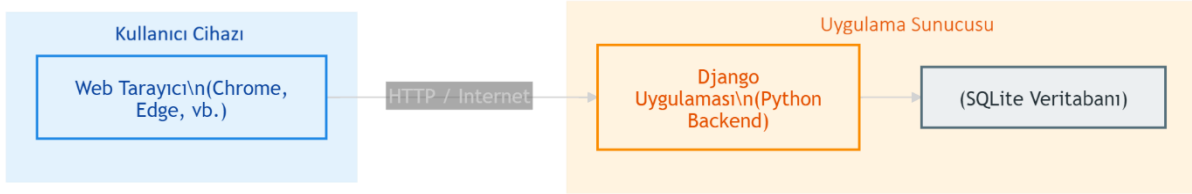


Figure 2: Sistem Yerleşim (deployment) Diyagramı

Figure 2’de sistemin fiziksel dağılımı gösterilmektedir. Kullanıcılar web tarayıcısı üzerinden internet aracılığıyla Django uygulamasına erişmektedir. Uygulama, verileri SQLite veritabanında saklamaktadır.

2.3 Kalıcı Veri Yönetimi

Online Müzayede Sistemi’nde kullanıcılar, ürünler, müzayedeler ve teklifler gibi verilerin kalıcı olarak saklanması kritik öneme sahiptir. Bu bağlamda sistemde veri bütünlüğünü, güvenliğini ve işlemlerin geriye dönük izlenebilirliğini sağlamak amacıyla ilişkisel bir veritabanı yönetim sistemi (RDBMS) kullanılacaktır. Sistemde kalıcı olarak saklanacak temel veri grupları şunlardır:

- **Kullanıcı Verileri:** Sisteme kayıt olan satıcı, müşteri ve yöneticilere ait T.C. Kimlik numarası, ad-soyad, e-posta, şifre ve rol bilgileri kalıcı olarak tutulacaktır. Her kullanıcının T.C. Kimlik ve e-posta bilgisinin benzersiz (unique) olması veritabanı kısıtlamaları ile güvence altına alınacaktır.
- **Ürün Verileri:** Satıcılar tarafından platforma eklenen ürünlerin adı, açıklaması, kategorisi, başlangıç fiyatı ve güncel onay statüleri (Taslak, Onay Bekliyor, Aktif, Satıldı, Reddedildi vb.) saklanacaktır.
- **Müzayede ve Teklif Verileri:** Aktif müzayedelerin bitiş zamanları, o anki mevcut en yüksek fiyatları ile müşteriler tarafından sunulan tekliflerin miktarı ve zaman damgaları (timestamp) kaydedilecektir. Tüm teklif ve satış işlemleri veritabanında loglanarak tam bir şeffaflık ve takip edilebilirlik sağlanacaktır.

Bu veri yapıları arasındaki temel ilişkisel veritabanı kurgusu şu şekildedir:

- Bir kullanıcı birden fazla ürün ekleyebilir (1-N ilişki)
- Bir ürün yalnızca bir müzayedeye sahiptir (1-1 ilişki)
- Bir müzayede birden fazla teklif içerir (1-N ilişki)
- Her teklif bir kullanıcıya aittir (N-1 ilişki)

Veri Erişim Stratejisi ve Veritabanı Seçimi

Geliştirme sürecinde temel altyapı olarak Django web framework’ünün varsayılan ilişkisel veritabanı olan SQLite kullanılacaktır. SQLite, sunucu kurulumu gerektirmeyen yapısıyla hızlı geliştirme imkânı sunarken, ACID (Atomicity, Consistency, Isolation, Durability) prensiplerini destekleyerek verilerin güvenli bir şekilde tablolar halinde ilişkisel olarak saklanmasını sağlayacaktır. Veritabanı işlemleri (CRUD - Create, Read, Update, Delete) veri erişim stratejisi

olarak doğrudan Django ORM (Object-Relational Mapping) üzerinden yönetilecektir. Bu sayede veritabanı işlemleri doğrudan SQL yazmadan, güvenli ve nesne tabanlı olarak yürütülebilecektir.

Sistemin veri tabanı mimarisini ve yukarıda bahsedilen ilişkileri görselleştiren Varlık-İlişki (ER) Diyagramı aşağıda *Figure 3*'te sunulmuştur.

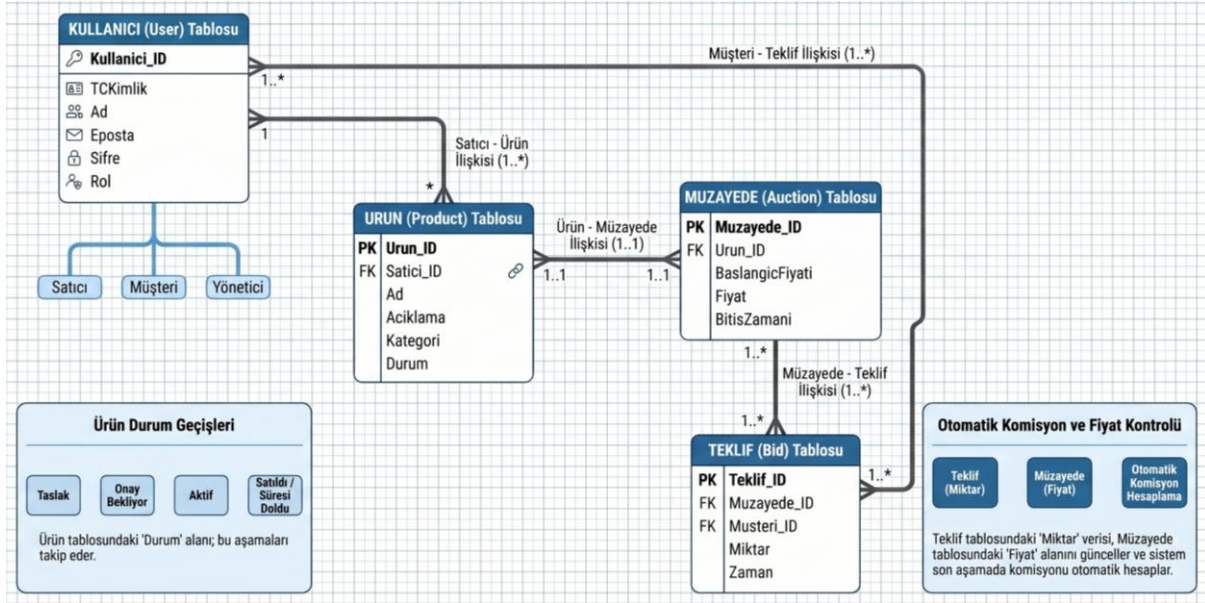


Figure 3: Veritabanı Varlık-İlişki (ER) Diyagramı

2.4. Erişim Kontrolü ve Güvenlik

BidLance platformunda, veri güvenliğini sağlamak ve sistem işleyişini korumak amacıyla Rol Bazlı Erişim Kontrolü (RBAC) mekanizması uygulanacaktır. Sisteme giriş yapan kullanıcılar, veritabanında tanımlı olan yetkilerine (Saticı, Müşteri, Yönetici) göre sadece kendi rollerine özel panellere otomatik olarak yönlendirileceklerdir.

Güvenlik ve Veri Koruma Önlemleri:

- **Kimlik Doğrulama (Authentication):** Kullanıcıların hesaplarına erişimi e-posta ve şifre doğrulaması ile sağlanacak olup, şifreler veritabanında açık metin (plaintext) yerine güvenli bir şekilde şifrelenerek (hashlenerek) saklanacaktır. Django'nun hazır authentication sistemi kullanılır.
- **Yetkilendirme Kontrolü (Authorization):** Yetkisiz kullanıcıların farklı rollere ait sayfalara ve işlemlere erişimi sistem tarafından engellenecektir. Örneğin, bir müşteri yönetici işlemlerine kesinlikle erişemeyecektir.
- **İşlem Bütünlüğü:** Müzayede sürecinin güvenilirliği için; ardışık teklif verme, süresi dolmuş bir ürüne teklif iletme veya başlangıç fiyatının/artış adımının altında teklif verme gibi kural dışı işlemler anlık olarak sistem tarafından denetlenip reddedilecektir.

Kullanıcı rollerinin sistem fonksiyonları üzerindeki yetkileri aşağıdaki matriste (*Figure 4*) özetlenmiştir:

Modül / İşlev	Satıcı	Müşteri	Yönetici
Hesap ve Profil Yönetimi	✓	✓	✓
Ürün Ekleme ve Güncelleme	✓	✗	✗
Müzayede İnceleme ve Arama	✓	✓	✓
Teklif Verme (Bidding)	✗	✓	✗
Ürün Onaylama / Reddetme	✗	✗	✓
Komisyon Oranı Belirleme	✗	✗	✓
Müzayede Sonlandırma (Manuel)	✗	✗	✓

Figure 4: Erişim Kontrol Matrisi

2.5. Küresel Kontrol Akışı

BidLance Online Müzayede Sistemi, merkezi bir kontrol yapısına sahip olup web tabanlı bir mimaride çalışır. Sistemdeki olay işleme süreci hem kullanıcı girişleri, teklif doğrulama gibi senkron hem de bildirim gönderimi, zamanlayıcı tabanlı müzayede sonlandırma gibi asenkron mekanizmaları içerir.

Kontrol Yapısı: Kontrol akışı, Kullanıcı Arayüzü , Sistem Katmanı, Veritabanı arasındaki koordinasyonla sağlanacaktır.

Alt Sistem Koordinasyonu: Sistem, her teklif girişinde muzayedeKontrol() metodunu çalıştırarak veritabanından güncel durumu sorgular. Teklif geçerli olduğunda Veritabanı güncellenirken, eş zamanlı olarak Bildirim Servisi tetiklenerek önceki teklif sahibine asenkron olarak "Teklifiniz Geçildi" uyarısı gönderilir.

Otomasyon: Müzayede bitiş zamanının takibi ve komisyon hesaplamaları, insan müdahalesine gerek kalmadan sistem tarafından merkezi olarak yönetilir.

2.6. Sınır Koşulları

Sistem, olağandışı durumları ve süreç geçişlerini yönetmek için tanımlanmış durum diyagramları ve hata yakalama mekanizmalarını kullanır.

2.6.1. Başlatma (Initialization)

Sistem Başlangıcı: Uygulama açıldığında kullanıcıyı karşılayan ilk durum Giriş/Kayıt ekranıdır. Sistemin işlevsel hale gelmesi için oturumOlustur() metodu ile geçerli bir kullanıcı oturumunun başlatılması gerekir.

Veri Yükleme: Başlatma aşamasında sistem, veritabanından aktif müzayedeleri ve kategorileri yükleyerek kullanıcıya sunar.

Ürün Başlatma: Bir ürünün sistem sınırlarına dahil edilmesi "Taslak" durumuyla başlar; ancak müzayedenin teknik olarak başlatılması için yöneticinin urunOnayla() komutunu vererek ürün durumunu "Aktif" hale getirmesi şarttır.

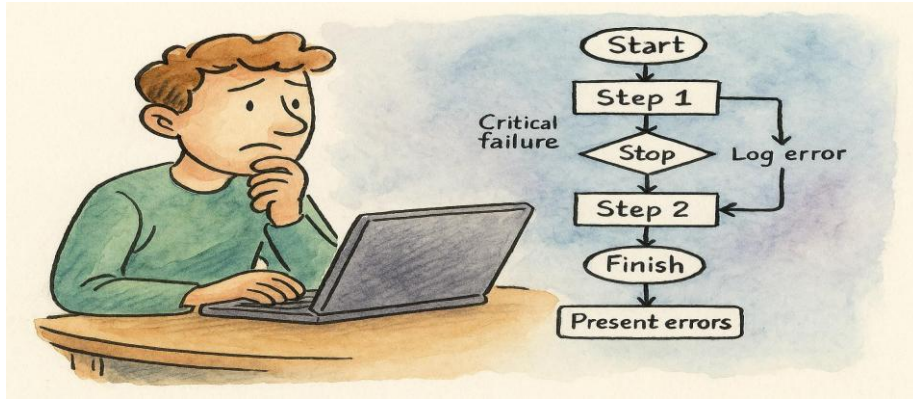
2.6.2. Sonlandırma (Termination)

Müzayede Sonu: Sistem, sureKontrol() mekanizması aracılığıyla bitiş zamanı gelen müzayedeleri otomatik olarak tespit eder ve teklif alımını kapatır.

Veri Güvenliği ve Kapatma: Sonlandırma anında kazananBelirle() fonksiyonu çalıştırılarak en yüksek teklif veritabanına kalıcı olarak işlenir. Aktif oturumlar kullanıcı tarafından cikisYap() veya oturumSonlandir() komutlarıyla güvenli bir şekilde kapatılır.

Durum Güncelleme: İşlem sonunda veri kaybını önlemek adına finansal kayıtlar (komisyonHesapla) oluşturulur ve ürün durumu "Satıldı" veya teklif yoksa "Süresi Doldu" olarak mühürlenir.

2.6.3. Hata Durumları (Error States)



Kural İhlalleri: Artış adımına uymayan teklifler, süresi dolmuş ürüne pey verme girişimi veya aynı kullanıcının üst üste teklif vermesi durumunda sistem işlemi durdurur ve kullanıcıya arayüz üzerinden anlık geri bildirim sağlar.

Doğrulama Hataları: Kayıt sırasında mükerrer T.C. Kimlik veya e-posta girilmesi durumunda "Hata Mesajı Göster" aktivitesi devreye girer.

Hata Telafi ve Kayıt: Teknik veya mantıksal hatalar oluştuğunda sistem, hataKaydet() ve islemKaydet() metodlarını kullanarak bu durumları loglar; böylece yöneticinin sistem sağlığını denetlemesine olanak tanır. Ağ kesintisi gibi durumlarda, veritabanı bütünlüğünü korumak için işlem gerçekleşmeden kullanıcıya uyarı iletir.

3. ALT DÜZEY TASARIM

3.1. Nesne Tasarım Kararları

Bu bölümde, Online Müzayede Sistemi'nin tasarımı aşamasında karşılaşılan temel mühendislik problemleri ve sistemin gereksinimlerini karşılamak adına verilen tasarım kararları (ödüneleşimler) detaylandırılmıştır.

1. Veri Tutarlılığı (Data Consistency) vs. Sistem Performansı

Müzayede sistemlerinde en kritik işlev, birden fazla Müşterinin aynı Ürün için eşzamanlı olarak *teklifVer()* metodunu tetiklemesidir (UC-08). Bu noktada sistemin hızlı yanıt vermesi (performans) ile tekliflerin doğru sırayla ve güvenli işlenmesi (veri tutarlılığı) arasında bir ödüneleşim yapılmıştır.

- **Karar:** Projemizde **Veri Tutarlılığı** tercih edilmiştir.
- **Gerekçe:** Müzayede güvenilirliğinin sağlanması için iki kullanıcının aynı anda en yüksek teklifi verdiğini düşünmesi kabul edilemez bir hatadır. Bu nedenle, *Müzayede* nesnesi içindeki fiyat güncelleme ve kazanan belirleme işlemleri sırasında eşzamanlılık (concurrency) kontrolleri ve veri tabanı kilitleme (locking) mekanizmaları kullanılacaktır. Bu durum sistemde milisaniyelik gecikmelere (performans kaybına) yol açabilecek olsa da, finansal verilerin %100 doğru ve güvenli işlenmesini garanti edecektir.

2. Geliştirilebilirlik/Esnelik vs. Geliştirme Süresi ve Karmaşıklık

Sistemde yeni bir teklif verildiğinde veya müzayede sonlandığında ilgili Müşterilere ve Satıcılara anlık bilgilendirme yapılması gerekmektedir (UC-09: *Bildirim Gönder*). Bu yapıyı doğrudan sınıflar içine gömmek kodlama süresini kısaltacak olsa da sistemi birbirine bağımlı (tightly coupled) hale getirecektir.

- **Karar:** Projemizde mimari karmaşıklık göze alınarak **Esnelik ve Genişletilebilirlik** tercih edilmiştir.
- **Gerekçe:** Bildirim sisteminin yönetimi için nesneler arası doğrudan çağrılar yapmak yerine, Observer (Gözlemci) vb. olay güdümlü (event-driven) bir mimari yaklaşım benimsenmiştir. Bu karar, başlangıçtaki kodlama süresini ve mimari karmaşıklığı artırsa da; ileride sisteme SMS, Mobil Push Bildirimi veya E-posta modülleri eklenmek istendiğinde Kullanıcı veya Müzayede çekirdek sınıflarının kodlarında hiçbir değişikliğe gerek kalmadan (Open-Closed prensibine uygun olarak) entegrasyon yapılmasına olanak tanıyacaktır.

3. İzlenebilirlik ve Güvenlik vs. Veritabanı Yüğü (Depolama)

Bir müzayedede (UC-12: *Müzayede Sonlandır*) sadece en yüksek teklifi tutmak, veritabanı yükünü ve nesne boyutlarını minimumda tutmak için cazip bir yöntemdir. Ancak sistemin güvenilirliğini denetleyecek olan *Yönetici* sınıfının şeffaflık ihtiyacı bulunmaktadır.

- **Karar:** Depolama maliyeti ve veritabanı yükü göze alınarak **Maksimum İzlenebilirlik (Traceability)** tercih edilmiştir.
- **Gerekçe:** Sadece anlık *mevcutFiyat* verisini tutmak yerine, bağımsız bir *Teklif* sınıfı tasarlanmıştır. Her bir teklif hamlesi; miktarı, zaman damgası ve müşterisiyle birlikte ayrı bir nesne olarak veritabanına kaydedilecektir (1..* çokluk ilişkisi). Bu durum, sunucu tarafında ciddi bir veri birikimine ve depolama maliyetine yol açacak olsa da, olası itirazlarda geçmişe dönük logların incelenebilmesini ve sistemdeki şüpheli fiyat manipülasyonlarının Yönetici tarafından tespit edilebilmesini sağlayacaktır.

3.2. Son Nesne Tasarımı

Bu bölümde, sistemin kavramsal analiz aşamasından tasarım aşamasına geçişi detaylandırılmış ve uygulanacak kod mimarisinin (Python) temel iskeleti oluşturulmuştur. Sistemin veri tutarlılığını ve güvenliğini sağlamak amacıyla nesne yönelimli programlama (OOP) prensipleri uygulanmış; sınıfların sahip olduğu veri yapıları ve metotlar kesinleştirilmiştir. Kapsülleme (encapsulation) kuralı gereği, doğrudan erişime kapalı olan özellikler *private* (-), dışarıdan çağrılacak işlemler ise *public* (+) erişim belirleyicileri ile tanımlanmıştır. Aşağıdaki UML sınıf diyagramı (*Figure 5*), sistemdeki soyut temel sınıfları, kullanıcı rollerini ve sistem varlıklarını güncel veri tipleriyle birlikte göstermektedir.

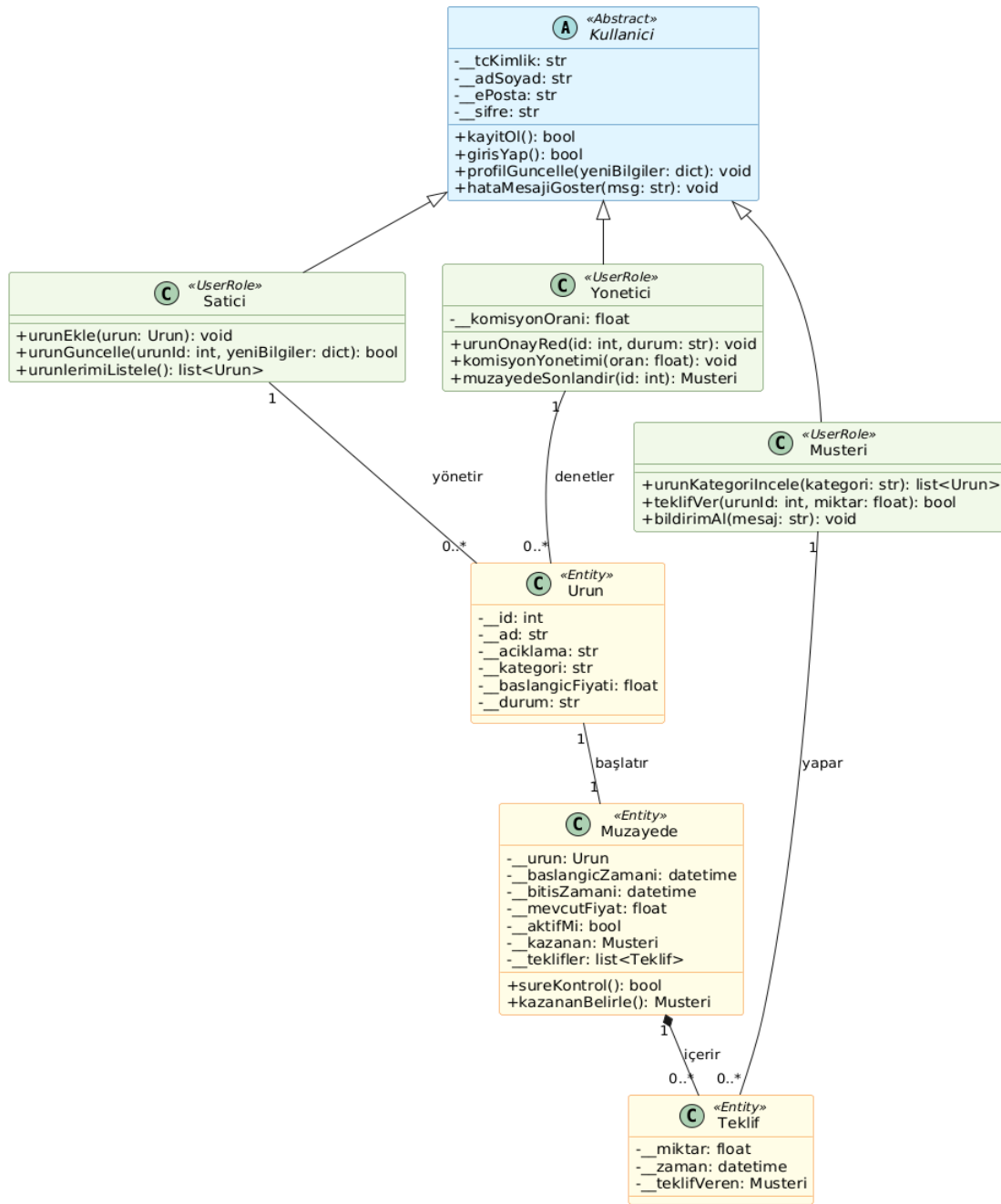


Figure 5: UML Sınıf Diyagramı

3.3. Paketler / Katmanlar

Sistem mimarisi, nesne yönelimli programlama (OOP) prensiplerine ve Python tabanlı Django framework'ünün sunduğu Model-View-Template (MVT) yapısına uygun olarak tasarlanmıştır. Sistemdeki olay işleme süreci, Kullanıcı Arayüzü, Sistem Katmanı ve Veritabanı arasındaki koordinasyonla sağlanmaktadır.

3.3.1. Dış Paketler

Sistemin güvenli, anlık ve çok kullanıcıli yapısını desteklemek için aşağıdaki teknolojiler entegre edilmiştir:

- **Django (Python):** Sistemin ana web framework'ü olarak kullanılmaktadır. Rol tabanlı erişim kontrolü (RBAC), oturum yönetimi ve nesne ilişkisel eşleme (ORM) işlemlerini yürütür.
- **SQLite:** Sistemin ilişkisel veritabanıdır. Eşzamanlı teklif durumlarında veri tutarsızlığını önlemek amacıyla veritabanı seviyesinde kilitleme (locking) mekanizmalarını çalıştırmak için kullanılır.
- **Django Channels (WebSocket):** Teklif güncellemelerinin senkron kontroller sonrasında asenkron olarak diğer kullanıcılara anlık iletilmesi ("Teklifiniz Geçildi" uyarıları) işlevini üstlenir.

3.3.2. İç Paketler (Uygulama - App Yapısı)

Sistem mimarisi, bakımın kolaylaşması ve yeniden kullanılabilirlik amacıyla Django'nun "app" yapısı esas alınarak modüler şekilde bölünmüştür.

- **Kullanıcı Yönetimi Paketi (User App):** Sisteme giriş (oturumOlustur), çıkış (cikisYap / oturumSonlandir), rol tespiti ve profil güncelleme işlemlerini barındırır.
- **Ürün Yönetimi Paketi (Product App):** Ürünlerin sisteme taslak olarak eklenmesi, yöneticinin urunOnayla() komutu ile sınır koşullarına dahil edilmesi ve durum güncellemelerini içerir.
- **Teklif ve Müzayede Paketi (Auction App):** Maksimum izlenebilirlik (traceability) kararımız gereği, her teklifin zaman damgasıyla ayrı bir nesne olarak işlendiği, muzayedeKontrol() ve sureKontrol() gibi merkezi otomasyon metotlarını barındıran çekirdek pakettir.

3.4. Sınıf Arayüzleri

Sistemdeki sınıflar; veriyi tutan "Entity (Varlık)", iş mantığını ve servisleri yürüten "Control (Kontrol)", ve kullanıcı ile etkileşimi sağlayan "Boundary (Sınır/Arayüz)" sınıfları olmak üzere üç ana kategoride tasarlanmıştır. Kapsülleme (encapsulation) kuralı gereği doğrudan erişime kapalı olan özellikler *private* (-), dışarıdan çağrılacak işlemler ise *public* (+) erişim belirleyicileri ile tanımlanmıştır.

3.4.1. Entity (Varlık) Sınıfları

Sistemin ana verilerini tutan ve veritabanı (model) katmanını temsil eden temel sınıflardır.

1. Kullanici (Abstract Class) Sistemdeki tüm aktörlerin ortak niteliklerini ve metotlarını barındıran temel/soyut sınıftır.

- **Öznitelikler (Attributes):**
 - -tcKimlik: String : Kullanıcının benzersiz kimlik numarası.
 - -adSoyad: String : Kullanıcının tam adı.
 - -ePosta: String : Sisteme giriş ve bildirimler için benzersiz e-posta adresi.

- -sifre: String : Güvenli şifre.

- **Metotlar (Methods):**

- +kayitOl(tc: String, ad: String, mail: String, sifre: String): Boolean → Sisteme yeni kayıt oluşturur. T.C. ve E-posta benzersizliğini denetleyerek duruma göre True/False döner.
- +girisYap(mail: String, sifre: String): Boolean → Kullanıcının e-posta ve şifresiyle oturum açmasını sağlar, yetki rolünü tespit eder.
- +profilGuncelle(bilgiler: Object): Void → Kullanıcının iletişim veya şifre bilgilerini günceller.
- +hataMesajiGoster(msg: String): Void → Kural ihlallerinde kullanıcıya uyarı yansıtır.

2. Satıcı (Kullanıcı'dan Türetilmiş Sınıf) Ürün yönetimi yetkilerine sahip kullanıcı sınıfıdır.

- **Metotlar:**

- +urunEkle(u: Urun): Void → Yeni ürünü sisteme "Taslak" veya "Onay Bekliyor" statüsü ile kaydeder.
- +urunGuncelle(id: int, guncelVeriler: Object): Void → Sadece müzayede süreci başlamamış ürünlerin bilgilerini (görsel, açıklama) düzenler.
- +urunlerimiListele(): List<Urun> → Satıcıya ait ürünlerin listesini (onay bekleyen, aktif, satılan vb.) döndürür.

3. Musteri (Kullanıcı'dan Türetilmiş Sınıf) Aktif katılım sağlayan ve teklif veren kullanıcı sınıfıdır.

- **Metotlar:**

- +urunKategoriIncele(kategoriAd: String): List<Urun> → İlgili kategoriye ait, durumu "Aktif" olan onaylanmış müzayedeleri filtreleyip listeler.
- +teklifVer(uId: int, tutar: double): Boolean → Müşterinin fiyat artırımını isteğini iletir. Kurallara uygunsa (artış adımı vb.) işlemi başlatır.
- +bildirimAl(msg: String): Void → "Teklifiniz Geçildi" gibi sistemden gelen anlık asenkron bildirimleri yakalar.

4. Yönetici (Kullanıcı'dan Türetilmiş Sınıf) Sistemin denetimini ve finansal ayarlarını yürüten sınıftır.

- **Öznitelikler:**

- -komisyonOrani: Double : Satışlardan kesilecek varsayılan sistem kesinti oranı.

- **Metotlar:**

- +urunOnayRed(id: int, statu: bool): Void → Onay bekleyen ürünü kurallara uygunluğuna göre "Aktif" ya da "Reddedildi" durumuna geçirir.
- +komisyonYonetimi(oran: double): Void → Finansal modüldeki komisyon oranını günceller.
- +muzayedeSonlandir(id: int): Void → İstisnai durumlarda müzayedeyi manuel olarak bitirir.

5. Urun (Entity) Müzayedeye konu olan malzemenin verisini tutar.

- **Öznitelikler:**

- -id: int : Ürünün benzersiz kayıt numarası.
- -ad: String, -aciklama: String, -kategori: String : Ürün detayları.
- -baslangicFiyati: Double : Ürünün minimum açılış fiyatı.
- -durum: String : Ürünün güncel yaşam döngüsü durumu (Örn: Taslak, Onay Bekliyor, Aktif, Satıldı, Reddedildi, Süresi Doldu).

6. Muzayede (Entity) Ürüne bağlı olan açık artırma takvimini ve anlık fiyatı temsil eder.

- **Öznitelikler:**

- -bitisZamani: DateTime : Müzayedenin kapanacağı kesin tarih ve saat.
- -mevcutFiyat: Double : Anlık olarak ulaşılan en yüksek geçerli teklif tutarı.

- **Metotlar:**

- +sureKontrol(): Boolean → Sistemin o anki saati ile bitisZamani değerini karşılaştırarak müzayedenin bitip bitmediğini döner.
- +kazananBelirle(): Musteri → Bitiş süresi geldiğinde, bağlı olduğu Teklif listesinden durumu "Geçerli" olan en yüksek teklif sahibini bulur.

7. Teklif (Entity) Her bir hamleyi veritabanında saklayan (Traceability/İzlenebilirlik) nesnedir.

- **Öznitelikler:**

- -miktar: Double : Müşterinin önerdiği fiyat.
- -zaman: DateTime : Teklifin yapıldığı anın zaman damgası.

3.4.2. Control (Kontrol / İş Mantığı) Sınıfları

Sistemin merkezinde yer alan; doğrulama, iletişim, hesaplama ve veritabanı koordinasyonunu sağlayan, sıra (sequence) diyagramında tanımlanan sınıflardır.

1. Sistem / Sistem Kontrol Servisi Senkron ve asenkron mekanizmaları yöneten ana orkestrasyon sınıfıdır.

- **Metotlar:**

- +muzayedeKontrol(uId: int, tutar: double): Boolean → Her teklif girişinde çalışarak eşzamanlılığı (locking) kontrol eder, teklifin geçerli olup olmadığını artış adımı kurallarına göre sınar.
- +hataKaydet(hataTuru: Exception): Void → Teknik sorunlarda yönetici için hata loglarını kaydeder.
- +islemKaydet(islem: String): Void → Veri tutarlılığını sağlamak için tüm kritik hareketleri (log) sisteme işler.

2. Bildirim Servisi (Observer Modülü) Olay güdümlü (event-driven) çalışan asenkron bilgilendirme sınıfıdır.

- **Metotlar:**

- +oncekiKullaniciyaBildirim(teklifId: int): Void → Yeni bir geçerli teklif girildiğinde tetiklenir ve statüsü "Geçildi" olan önceki müşteriye asenkron uyarı gönderir.
- +MusteriyeBilgi(sonuc: String): Void & +SaticiyeBilgi(sonuc: String): Void → Müzayede sonlandığında alıcıya ve satıcıya işlem sonucunu iletir.

3. Odeme / Finans Servisi Süreci sonlanan müzayedelerde maddi hesaplamaları yapan sınıftır.

- **Metotlar:**

- +komisyonHesapla(satisTutari: double, komisyonOrani: double): Double → İnsan müdahalesi olmadan kazanan teklif üzerinden komisyon kesintisini hesaplar ve satıcıya aktarılacak net tutarı geri döner.

4. Veritabanı Servisi (Repository) Sistem ile kalıcı hafıza arasındaki CRUD (Oluşturma, Okuma, Güncelleme, Silme) operasyonlarını üstlenir.

- **Metotlar:**

- +urunuKaydet(durum: String, u: Urun): Void → Yeni ürünü belirtilen statü ile kaydeder.
- +durumGuncelle(id: int, yeniDurum: String): Void → Ürün "Onay Bekliyor"dan "Aktif"e veya "Satıldı"ya geçtiğinde tabloları günceller.
- +teklifiKaydet(t: Teklif): Void → Doğrulanmış yeni teklifi zaman damgası ile sisteme yazar.

3.4.3. Boundary (Arayüz / Sınır) Sınıfları

Kullanıcı ile arka plandaki iş mantığı sistemini birbirinden soyutlayan, kullanıcı isteklerini (HTTP Requests) algılayıp Sistem kontrolcüsüne ileten sınıftır.

1. Web Arayüzü (Frontend / View Router)

- **Metotlar (Kullanıcı İstek Yakalayıcılar):**

- +urunEkle(bilgiler: Object): Void → Kullanıcının HTML formunda doldurduğu ürün verilerini JSON/Nesne formatında yakalayıp Sistem'e iletir.
- +onayBekleyenUrunler(): List<Urun> → Yöneticinin arayüzde görmek istediği onaysız ürünlerin listesini talep eder ve UI'da (ekranda) render eder.
- +urunOnayla(urunID: int): Void → Yönetici onay butonuna bastığında kimlik numarasını iş mantığına aktarır.
- +teklifVer(urunID: int, teklifMiktari: double): Void → Müşteri teklif formunu gönderdiğinde verileri yakalar ve işlemin başarısına/başarısızlığına göre arayüzde doğru modalı/uyarayı gösterir.

3.5. Tasarım Desenleri

Sistem gereksinimlerini ve alınan mimari ödünleşim (trade-off) kararlarını desteklemek için aşağıdaki tasarım desenleri uygulanmıştır:

- **Observer (Gözlemci) Deseni / Event-Driven Mimari:**

- *Kullanım Amacı:* Müşterilere "Teklifiniz Geçildi" bildirimi gönderilmesi gereksinimini karşılamak.
- *Neden Tercih Edildi:* Bildirim kodlarını Kullanıcı veya Müzayede çekirdek sınıflarına doğrudan gömmek yerine, sistemin birbirine sıkı sıkıya bağlı (tightly coupled) olmasını engellemek için tercih edilmiştir.
- *Uygulama:* Open-Closed prensibine uygun olarak; ileride sisteme SMS veya Mobil Push bildirimleri eklendiğinde, çekirdek kodlarda hiçbir değişiklik yapmadan kolayca entegre edilebilmesini sağlar. Bu yapı, teklif alındığında asenkron olarak tetiklenir.

- **State (Durum) Deseni:**

- *Kullanım Amacı:* Sistemdeki varlıkların sınır koşullarını ve geçişlerini yönetmek.
- *Uygulama:* Ürün nesnesi için "Taslak, Onay Bekliyor, Aktif, Satıldı, Reddedildi, Süresi Doldu" yaşam döngülerini; *Teklif* nesnesi için ise "Oluşturuldu, Kontrol Ediliyor, Geçerli, Geçersiz, Geçildi, Kazandı" durumlarını kod seviyesinde standartlaştırmak için kullanılmıştır.

- **RBAC (Role Based Access Control) Deseni:**

- *Kullanım Amacı:* Her kullanıcının (Satıcı, Müşteri, Yönetici) yalnızca kendi yetkili olduğu işlemleri yapabilmesini ve veri güvenliğini sağlamak. Giriş işlemi başarılı olduğunda sistem, kullanıcının rolünü tespit ederek ilgili panelin yüklenmesini ve yetkisiz erişimlerin engellenmesini koordine eder.

Sözlük (Glossary)

Concurrency (Eşzamanlılık): Birden fazla kullanıcının aynı anda sistem üzerinde işlem gerçekleştirmesi durumudur. Bu projede veri çakışmasını önlemek için kontrol mekanizmaları ile yönetilmektedir.

Data Consistency (Veri Tutarlılığı): Sistem içerisindeki verilerin her zaman doğru, güncel ve çelişkisiz olması durumudur.

Locking (Kilitleme Mekanizması): Eşzamanlı işlemlerde veri çakışmasını önlemek için belirli veri üzerinde geçici erişim kısıtlaması uygulanmasıdır.

Traceability (İzlenebilirlik): Sistemde yapılan tüm işlemlerin geriye dönük olarak takip edilebilmesi ve kayıt altına alınmasıdır.

OOP (Object-Oriented Programming / Nesne Yönelimli Programlama): Yazılımın nesneler ve sınıflar üzerinden modellenmesini sağlayan programlama yaklaşımıdır.

Encapsulation (Kapsülleme): Bir nesnenin verilerinin ve bu verilere erişimin kontrol altına alınması prensibidir.

RBAC (Role-Based Access Control): Kullanıcıların sistemdeki yetkilerinin rollerine göre sınırlandırılmasıdır (Satıcı, Müşteri, Yönetici).

ORM (Object-Relational Mapping): Veritabanı işlemlerinin nesneler üzerinden gerçekleştirilmesini sağlayan yapı (Django ORM).

ACID Prensipleri: Veritabanı işlemlerinde güvenilirliği sağlayan dört temel özellik: Atomicity, Consistency, Isolation, Durability.

CRUD İşlemleri: Create, Read, Update, Delete işlemlerinin genel adıdır.

WebSocket: İstemci ile sunucu arasında gerçek zamanlı veri iletişimi sağlayan teknolojidir.

Observer Design Pattern: Bir nesnede meydana gelen değişikliklerin diğer nesnelere otomatik olarak bildirilmesini sağlayan tasarım desendir.

State Design Pattern: Bir nesnenin durumuna göre davranış değiştirmesini sağlayan tasarım desendir.

Event-Driven Architecture (Olay Güdümlü Mimari): Sistem bileşenlerinin olaylara tepki vererek çalıştığı yazılım mimarisi yaklaşımıdır.

Backend: İş mantığının çalıştığı sunucu tarafı yazılım katmanıdır.

Frontend: Kullanıcının doğrudan etkileşimde bulunduğu arayüz katmanıdır.

Deployment (Dağıtım): Uygulamanın çalışır hale getirilerek kullanıcıya sunulması sürecidir.

Referanslar

- [1] Django Software Foundation, *Django Documentation*, [Online]. Available: <https://docs.djangoproject.com/>
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [4] GeeksforGeeks, *System Design - UML Introduction*, [Online]. Available: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>
- [5] SQLite Documentation, *SQLite Official Documentation*, [Online]. Available: <https://www.sqlite.org/docs.html>
- [6] Python Software Foundation, *Python Documentation*, [Online]. Available: <https://docs.python.org/3/>
- [7] PlantUML, *PlantUML Documentation*, [Online]. Available: <https://plantuml.com/>
- [8] MDN Web Docs, *WebSocket API*, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [9] IEEE, *IEEE Citation Guidelines*, [Online]. Available: <https://ieeauthorcenter.ieee.org/>